

Understanding Agile Methodology

1. Traditional project management vs. Agile

Before Agile existed, most IT projects followed what is often called the **Waterfall** approach. Imagine a real waterfall: water flows in one direction only, from the top down, and never goes back up. A Waterfall project works the same way:

1. Gather **all** the requirements at the very beginning.
2. Design the whole system.
3. Build the whole system.
4. Test everything at the end.
5. Deliver the finished product to the client.

The problem: if the client changes their mind in month 2, or if a requirement was misunderstood, the team usually doesn't find out until month 8 — when it is very expensive to fix. Nothing is shown to the client until the very end.

Agile flips this around. Instead of one long cycle, the project is split into many small cycles. At the end of each small cycle, something **working** is shown to the client, who can then give feedback before the next cycle starts.

Waterfall	Agile
Requirements frozen at the start	Requirements can evolve throughout the project
One big delivery at the end	Many small deliveries, every 1-4 weeks
Client feedback comes at the end	Client feedback comes constantly
Documentation-heavy	Working software over heavy documentation
Change is expensive and discouraged	Change is expected and welcomed

2. What Agile actually is

Agile is not a single tool or a strict recipe — it is a **set of values and principles** for organizing work, first written down in 2001 in the "Agile Manifesto". In plain English, the four core values are:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

How to remember it: Agile doesn't say tools, documentation, contracts, and plans are useless — it says that when you have to choose, people, working results, collaboration, and adaptability should win.

3. The main frameworks

"Agile" is the philosophy; **Scrum**, **Kanban**, and **Extreme Programming (XP)** are concrete frameworks that apply it.

Scrum

Work is organized into fixed-length iterations called **sprints** (usually 1 to 4 weeks). Each sprint has a beginning (planning), a middle (daily check-ins), and an end (a demo plus a team retrospective). Scrum defines three roles: Product Owner, Scrum Master, and the Development Team.

Kanban

Instead of fixed sprints, work flows continuously through columns on a board (e.g. *To Do* → *In Progress* → *Done*). There is a limit on how many tasks can be "In Progress" at once (called **WIP limit**, for Work In Progress), which prevents the team from starting too many things without finishing any of them.

Extreme Programming (XP)

XP is closer to Scrum but focuses more on engineering practices: pair programming (two developers, one keyboard), test-driven development, and very frequent releases.

Which one for which team? Scrum suits teams that need regular, predictable checkpoints with a client or manager. Kanban suits support/maintenance teams with a steady, unpredictable stream of small requests (e.g. a helpdesk). XP suits teams that want to push technical

4. Key Scrum vocabulary, explained simply

Term	Plain-English meaning
Product Backlog	The full to-do list for the whole product, ranked by priority.
Sprint	A short, fixed time box (e.g. 2 weeks) during which a chunk of work gets done.
Sprint Backlog	The smaller to-do list for just the current sprint.
Sprint Planning	The meeting where the team picks what goes into the sprint.
Daily Stand-up	A 15-minute daily check-in: what I did, what I'll do, what's blocking me.
Sprint Review	A demo of the finished work to stakeholders at the end of the sprint.
Sprint Retrospective	A private team meeting to discuss how to work better next time.
Definition of Done	The checklist a task must pass before it can be called "finished".
Velocity	How much work the team typically completes per sprint — used to plan future sprints.

5. Roles in a Scrum team

- Product Owner** Decides *what* gets built and in what order. Represents the customer/business.
- Development Team** Decides *how* to build it. Cross-functional (developers, testers, etc.) and self-organizing.
- Scrum Master** Makes sure the process runs smoothly and removes blockers. Not a boss — more of a coach/facilitator.

6. Why companies adopt Agile: real benefits

- **Faster feedback:** mistakes and misunderstandings are caught within weeks, not months.
- **Better adaptability:** priorities can change between sprints without wrecking the whole plan.
- **Higher visibility:** stakeholders see real progress at every sprint review, not just promises.
- **Improved morale:** teams have more ownership over how they organize their own work.

7. Common misconceptions

Misconception	Reality
"Agile means no planning."	Agile means planning constantly, in small steps, instead of once at the start.
"Agile means no documentation."	Documentation still exists — it's just not the main measure of progress.
"Agile means no deadlines."	Sprints are strict deadlines; they are just much shorter ones.
"Agile is only for software."	Marketing, HR, and even manufacturing teams use Agile/Kanban today.

8. A simple visual: the sprint cycle

Sprint Planning → Daily Stand-ups (repeated every day) → Sprint Review → Sprint Retrospective → *(back to Sprint Planning for the next sprint)*